Пам'ять у C++: вказівники, стек, купа, адресація, бітові операції

1. Пам'ять і адреси

Оперативна пам'ять — це довгий масив байтів. Кожен байт має свій номер, тобто **адресу**. Змінна займає один або кілька байтів поспіль, а її адресою вважають адресу першого байта.

```
int x = 42;
std::cout << &x;          // адреса x, наприклад 0x7ffd5c3a1b4c
std::cout << sizeof(x);   // 4 байти (на більшості платформ)
```

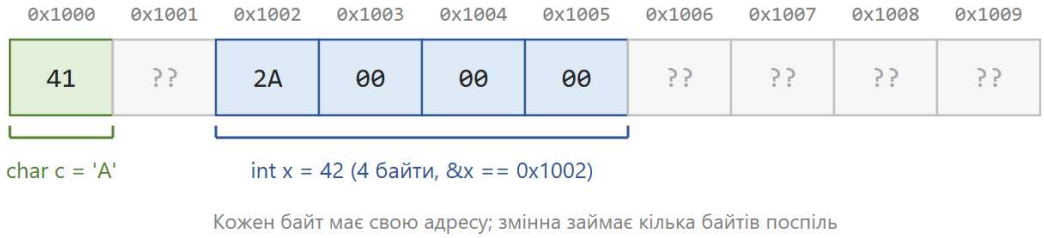


Рис. 1. Байти пам'яті та їхні адреси

-
-
-

Порядок байтів (endianness). На x86/ARM порядок little-endian: молодший байт лежить за меншою адресою. Число `0x12345678` в пам'яті виглядає так: `78 56 34 12`.



Рис. 2. Little-endian і big-endian

2. Вказівники

Вказівник — це змінна, яка зберігає адресу іншої змінної.

```
int  x = 10;
int* p = &x;    // p зберігає адресу x
*p = 20;        // розіменування: пишемо туди, куди вказує p
```

```
// тепер x == 20
```



Рис. 3. Вказівник p зберігає адресу x

Запис	Що означає
`int* p`	p — вказівник на int
`&x`	адреса x
`*p`	значення за адресою p
`p->field`	те саме, що `(*p).field`
`nullptr`	«нікуди не вказує»

Арифметика вказівників

Вказівник рухається кроками розміром у **свій тип**, а не в байти:

```
int a[4] = {10, 20, 30, 40};
int* p = a;        // ім'я масиву неявно перетворюється на &a[0]
p + 1;             // адреса більша на sizeof(int) = 4 байти
*(p + 2) == a[2];  // true: a[i] — це просто *(a + i)
```

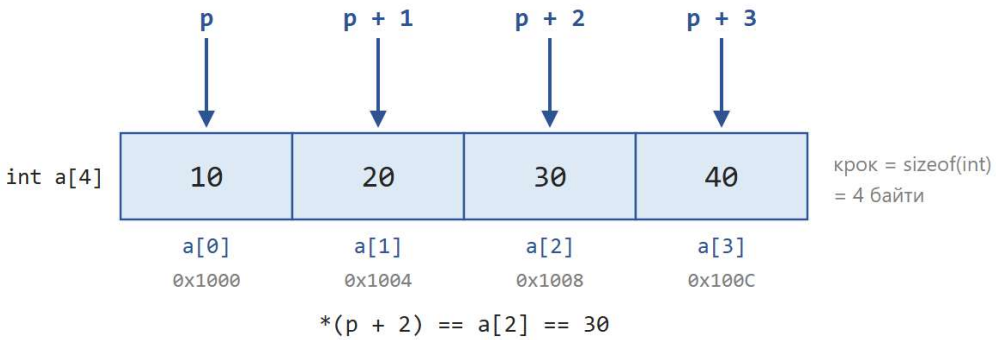


Рис. 4. Арифметика вказівників

const і вказівники

```
const int* p1;       // не можна змінити *p1 (дані сталі)
int* const p2 = &x;  // не можна змінити p2 (адреса стала)
```

Правило: читай справа наліво. `int* const p` — «p — сталий вказівник на int».

Посилання (reference)

Посилання — це інше ім'я для вже наявної змінної. Воно не буває порожнім і не переприв'язується. Під капотом це зазвичай той самий вказівник, тільки безпечніший.

```
void inc(int& v) { ++v; }   // змінює оригінал
void inc(int* v) { ++*v; }  // те саме через вказівник
```

3. Стек

Пам'ять процесу поділена на кілька областей. Стек і купа ростуть назустріч одне одному.

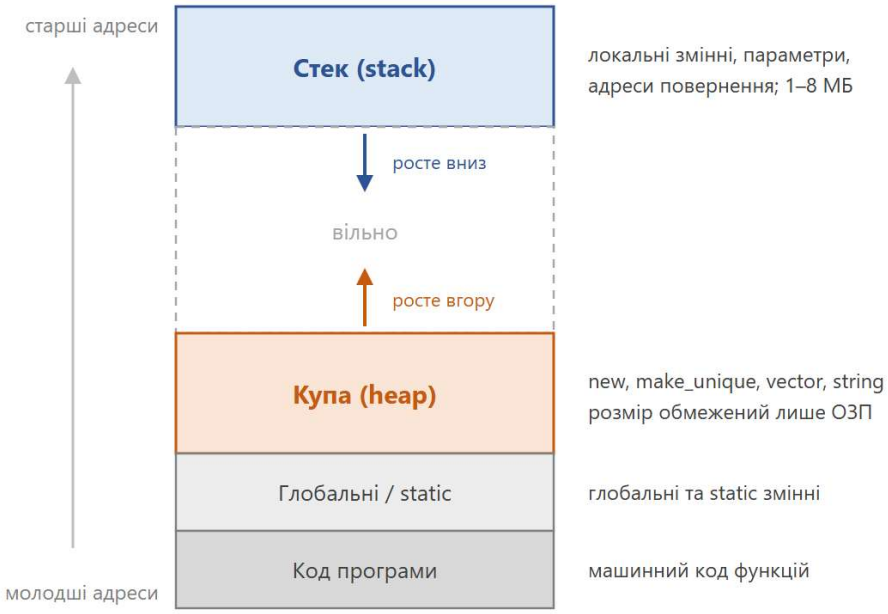


Рис. 5. Карта пам'яті процесу

Стек — область пам'яті для локальних змінних і викликів функцій.

- змінними та адресою повернення.
-
-
- дає **stack overflow**.

```
void f() {
```

```
    int a = 1;        // на стеку
    double arr[100];  // теж на стеку, 800 байтів
}                     // тут a та arr знищуються
```



Рис. 6. Кадри стека під час виклику функції

Класична помилка: повернути адресу локальної змінної.

```
int* bad() {
    int local = 5;
    return &local;    // висячий вказівник (dangling): кадр уже знищено
}
```

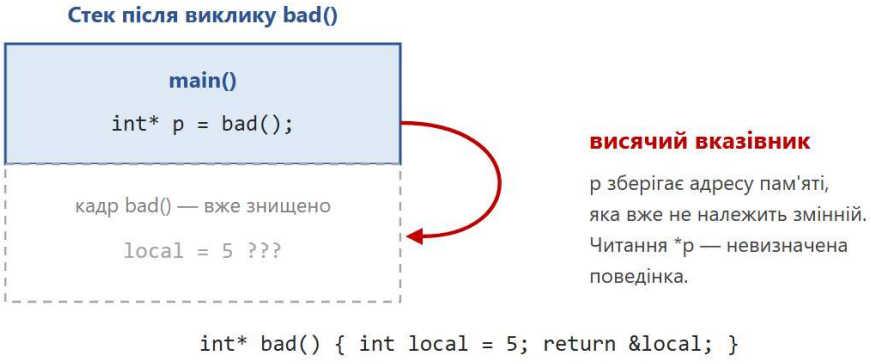


Рис. 7. Висячий вказівник

4. Купа (heap)

Купа — область для динамічної пам'яті, якою програміст керує сам.

-
-
-

```
int* p = new int(42);       // один int у купі
delete p;                   // звільнити
```

```
int* arr = new int[n];      // масив у купі
delete[] arr;               // для масиву обов'язково delete[]
```

Типові помилки

Помилка	Що відбувається
Витік пам'яті (leak)	`new` без `delete`, пам'ять не повертається
Double free	`delete` двічі для того самого вказівника
Use after free	звертання до пам'яті після `delete`
`delete` замість `delete[]`	невизначена поведінка

Сучасний C++: розумні вказівники

У сучасному коді `new`/`delete` вручну майже не пишуть. Ресурсом володіє об'єкт, а деструктор звільняє пам'ять автоматично (ідіома **RAII**).

```
#include <memory>
auto u = std::make_unique<int>(42);  // один власник, сам викличе delete
auto s = std::make_shared<int>(7);   // спільне володіння, лічильник посилань
std::vector<int> v(n);               // динамічний масив, теж у купі
```

Стек vs купа

	Стек	Купа
Хто керує	компілятор, автоматично	програміст (або RAII)
Швидкість	дуже швидко	повільніше
Розмір	малий, фіксований	великий, обмежений ОЗП
Час життя	до кінця блоку `{}`	до `delete` / деструктора
Типова помилка	stack overflow, dangling	leak, double free

Зверніть увагу: у `std::vector<int> v(1000);` сам об'єкт `v` (три вказівники) лежить на стеку, а його 1000 елементів лежать у купі.

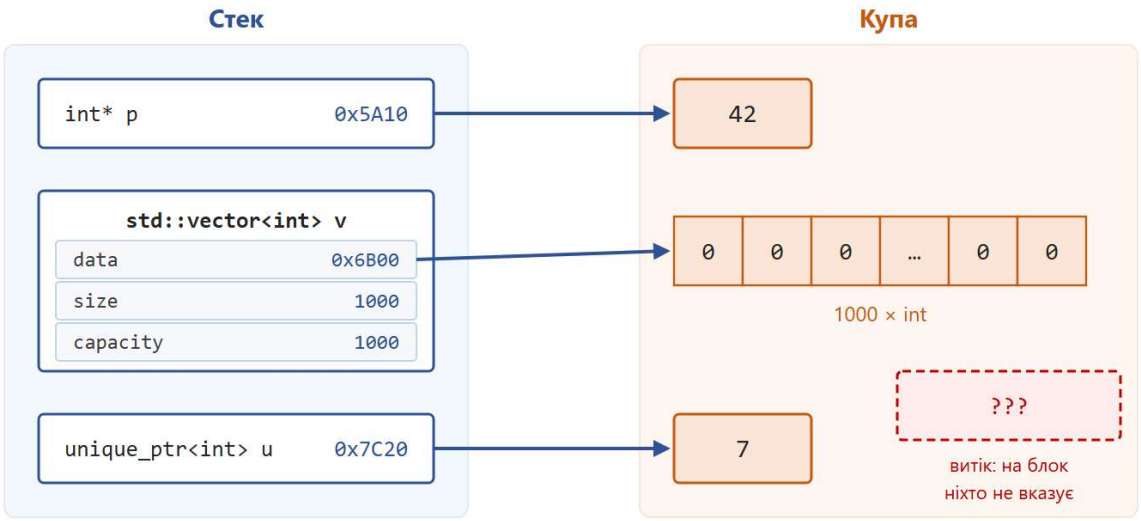


Рис. 8. Об'єкти на стеку вказують на дані в купі

5. Бітові операції

Працюють з цілими числами побітово.

Оператор	Назва	Приклад (a=0b1100, b=0b1010)	Результат
&	AND	`a & b`	`0b1000`
|	OR	`a | b`	`0b1110`
^	XOR	`a ^ b`	`0b0110`
~	NOT	~a (для 4 бітів)	`0b0011`
<<	зсув вліво	`a << 1`	`0b11000`
>>	зсув вправо	`a >> 2`	`0b0011`

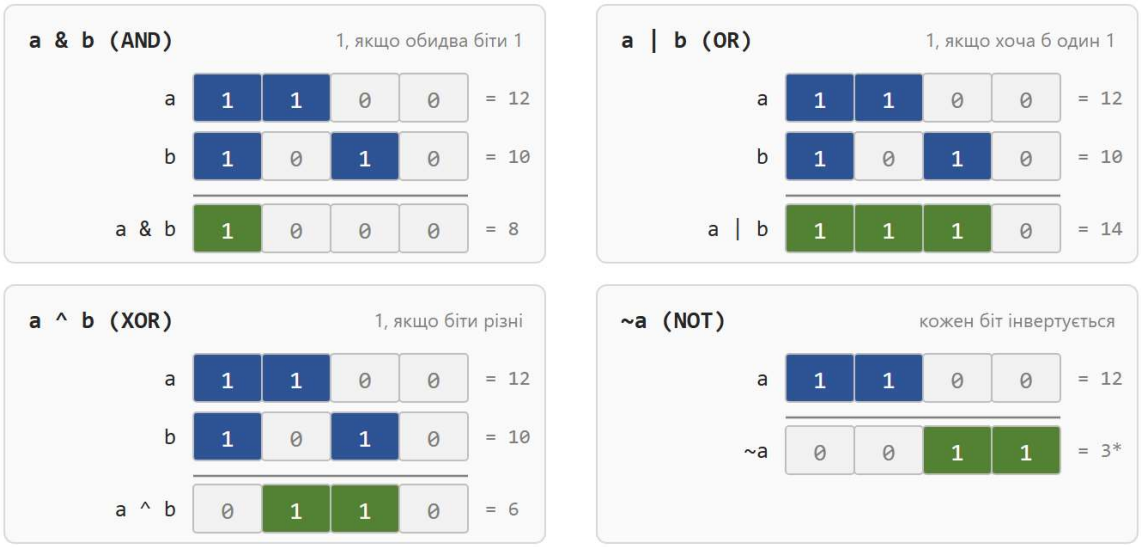


Рис. 9. AND, OR, XOR, NOT побітово

-
- зсув від'ємних чисел і переповнення знакових можуть бути визначені реалізацією або взагалі невизначені.



Рис. 10. Зсуви вліво і вправо

Як побачити біти: функція printBits

Щоб перевіряти бітові операції, зручно мати функцію, яка друкує число одразу у трьох виглядах: десятковому, двійковому і шістнадцятковому. Їй потрібен лише `cout`. Третій параметр каже, скільки бітів показати: 8 для `char`, 32 для `int`.

```
#include <iostream>
using namespace std;

// Друкує число: значення, біти (старший зліва, по 4) і hex.
// bits - скільки бітів показати: 8 для char, 32 для int.
```

```
void printBits(const char* name, long long value, int bits) {
    unsigned long long u = (unsigned long long)value;
    cout << name << " = " << value << "\t";
    for (int i = bits - 1; i >= 0; i--) {
        cout << ((u >> i) & 1);              // біт номер i
        if (i % 4 == 0 && i != 0) cout << ' ';
    }
    unsigned long long mask = (1ULL << bits) - 1;   // лише потрібні біти
    cout << "\t0x" << hex << (u & mask) << dec << endl;
}
```

Головний рядок функції — `(u >> i) & 1`. Зсув переносить біт номер i у наймолодшу позицію, а `& 1` відкидає всі інші біти. Цикл іде від старшого біта до молодшого, тому число друкується так, як ми його пишемо на папері.

Значення приймаємо як `long long`, щоб у функцію можна було передати і `char`, і `int`, і від'ємні числа. Приведення до `unsigned long long` не змінює бітів, воно лише дозволяє безпечно їх зсувати.

Використання:

```
unsigned char m = 12, n = 10;   // 0000 1100 і 0000 1010
printBits("a", m, 8);
printBits("b", n, 8);
printBits("a & b", m & n, 8);
printBits("a | b", m | n, 8);
printBits("a ^ b", m ^ n, 8);
printBits("~a", (unsigned char)~m, 8);
printBits("a << 1", m << 1, 8);
printBits("a >> 2", m >> 2, 8);
printBits("-88", -88, 8);       // додатковий код
printBits("-1", -1, 32);        // самі одиниці
```

Вивід:

```
a = 12      0000 1100   0xc
b = 10      0000 1010   0xa
a & b = 8   0000 1000   0x8
a | b = 14  0000 1110   0xe
a ^ b = 6   0000 0110   0x6
~a = 243    1111 0011   0xf3
a << 1 = 24 0001 1000   0x18
a >> 2 = 3  0000 0011   0x3
```

```
-88 = -88   1010 1000   0xa8
-1 = -1     1111 1111 1111 1111 1111 1111 1111 1111   0xffffffff
```

Зверніть увагу на кілька речей:

- тип беззнаковий, і всі 8 бітів просто інвертуються.
- а −88 у 8 бітах — це `1010 1000`.
-

Ідіоми роботи з бітом номер k

```
x |=  (1u << k);        // встановити біт
x &= ~(1u << k);        // скинути біт
x ^=  (1u << k);        // інвертувати біт
bool on = (x >> k) & 1u; // перевірити біт
```

Маска `1u << k` — це число з єдиною одиницею в позиції k. Операція з маскою зачіпає тільки цей біт і не чіпає решту.

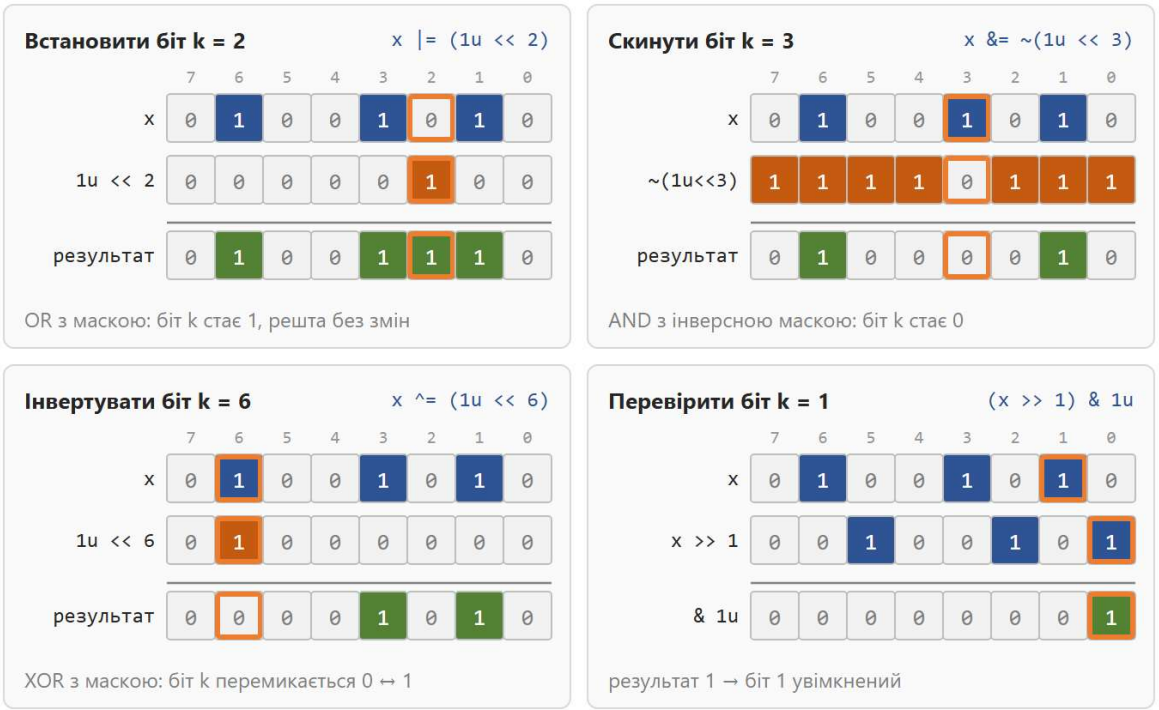


Рис. 11. Встановлення, скидання, інверсія і перевірка біта

Корисні трюки

```
(x & 1) == 0            // x парне
x & (x - 1)             // прибрати наймолодшу одиницю
x != 0 && (x & (x-1)) == 0  // x є степенем двійки
x & -x                  // залишити тільки наймолодшу одиницю
```

```
a ^= b; b ^= a; a ^= b;  // обмін без тимчасової змінної (лише як цікавинка)
```

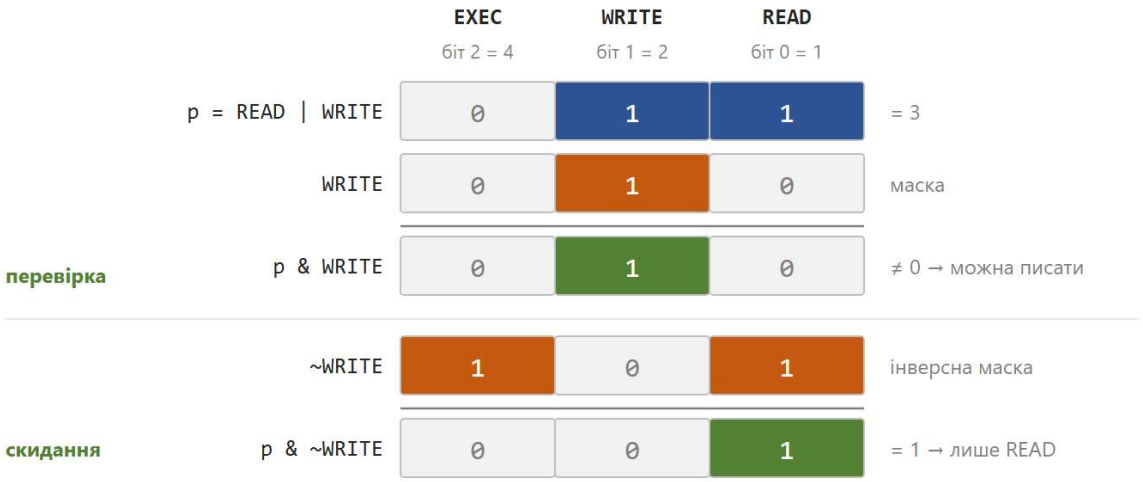


Рис. 12. Бітові трюки покроково

Бітові маски (прапорці)

```
const unsigned int READ = 1, WRITE = 2, EXEC = 4;   // біти 0, 1, 2

unsigned int perm = READ | WRITE;   // 0011
if (perm & WRITE) { /* можна писати */ }
perm = perm & ~WRITE;               // забрати право запису: 0001
```



Кожен біт — окремий прапорець «так/ні»: один unsigned вміщує 32 незалежні прапорці

Рис. 13. Прапорці в одному числі

Де бітові маски потрібні на практиці

Колір RGB. Колір часто зберігають в одному `unsigned int` як `0xRRGGBB`. Щоб дістати канал, зсуваємо потрібний байт у молодшу позицію і відрізаємо решту маскою `0xFF`.

```
unsigned int c = 0xFF8040;
unsigned int r = (c >> 16) & 0xFF;   // 255
unsigned int g = (c >> 8)  & 0xFF;   // 128
unsigned int b =  c        & 0xFF;   // 64
unsigned int back = (r << 16) | (g << 8) | b;   // зібрати назад
```

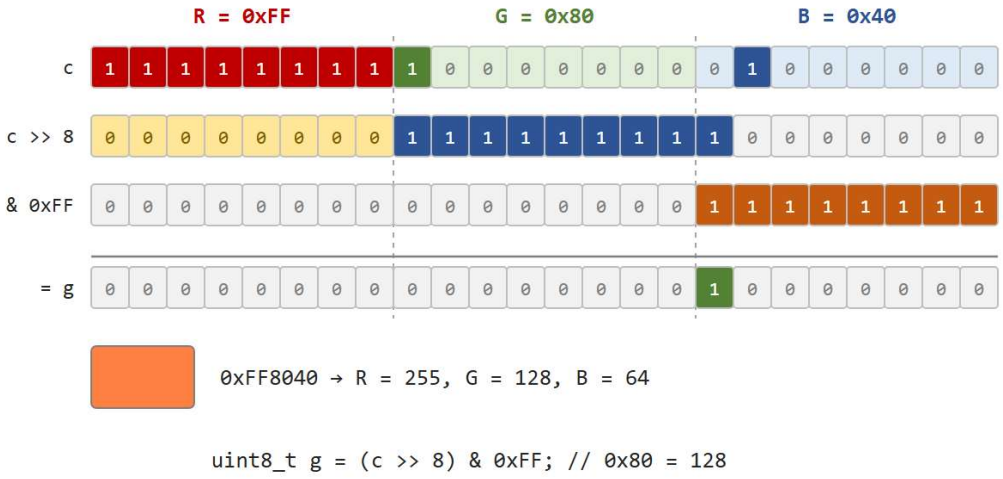


Рис. 14. Виділення каналу G з кольору

Маска підмережі. IP-адреса — це 32-бітне число. AND з маскою залишає номер мережі, тому за ним можна перевірити, чи два комп'ютери в одній мережі.

```
unsigned int ip   = (192u << 24) | (168u << 16) | (1u << 8) | 37u;  // 192.168.1.37
unsigned int mask = 0xFFFFFF00;                                     // 255.255.255.0 =
/24
unsigned int net  = ip & mask;                                      // 192.168.1.0
bool same = (ip & mask) == (other_ip & mask);
```



Рис. 15. IP-адреса AND маска підмережі

Права доступу в Unix. Права `rwx` для власника, групи та інших займають 9 бітів. Число 755 у вісімковій системі означає `rwxr-xr-x`.

```
unsigned mode = 0755;               // вісімковий літерал
bool owner_can_write = mode & 0200; // біт w власника
bool others_can_exec = mode & 0001; // біт x для інших
```

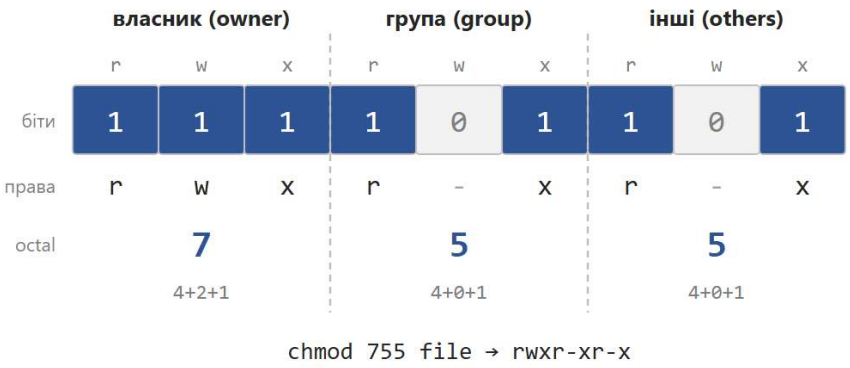


Рис. 16. Права доступу chmod 755

Ті самі прийоми використовують у регістрах мікроконтролерів, у мережевих заголовках, у форматах файлів і в шаховому рушії (bitboard: 64 клітинки = 64 біти).

6. Підсумок

1. Адреса — номер байта в пам'яті; `&` бере адресу, `*` розіменовує.
2. Вказівник зберігає адресу, а арифметика з ним рахує в розмірах типу.
3. Стек автоматичний і швидкий, але малий; не повертайте адреси локальних змінних.
4. Купа гнучка, але вимагає звільнення; у сучасному C++ це роблять `unique_ptr`, `shared_ptr`, `vector`.
5. Бітові операції дають компактні прапорці й швидкі трюки; працюйте з `unsigned`.

Запитання для самоперевірки

1. Що виведе `sizeof(int*)` на 64-бітній системі і чому?
2. Чим `const int* p` відрізняється від `int* const p`?
3. Де лежить об'єкт `std::string s = "hello";` і де його символи?
4. Чому функція `int* bad()` з розділу 3 некоректна?
5. Як одним виразом перевірити, що число є степенем двійки?
6. Як дістати синій канал з кольору `0xRRGGBB` і як перевірити, що дві IP-адреси в одній мережі /24?

Запуск у Visual Studio: українські літери в консолі

Щоб кирилиця в консолі відображалась правильно, потрібні дві речі.

1. Ключ компілятора `/utf-8`: Properties → C/C++ → Command Line → Additional Options. Він каже компілятору, що файл `.cpp` записаний в UTF-8.
2. Перемкнути консоль на UTF-8 на початку `main`:

```
#include <windows.h>
```

```
int main() {
    SetConsoleOutputCP(CP_UTF8);
    // ...
}
```

Робочий приклад до всіх розділів: demo.cpp.

Додаток. Повний приклад (demo.cpp)

Збірка у Visual Studio з ключем `/utf-8` або з командного рядка: `cl /EHsc /utf-8 demo.cpp`

```
// Демонстрація: адреси, вказівники, стек, купа, бітові операції.
// Visual Studio: Properties -> C/C++ -> Command Line -> Additional Options: /utf-8
// Командний рядок: cl /EHsc /utf-8 demo.cpp
#include <iostream>
#include <windows.h>   // SetConsoleOutputCP: щоб консоль показувала українські літери
using namespace std;

// Друкує число: значення, біти (старший зліва, по 4) і hex.
// bits - скільки бітів показати: 8 для char, 32 для int.
void printBits(const char* name, long long value, int bits) {
    unsigned long long u = (unsigned long long)value;
    cout << name << " = " << value << "\t";
    for (int i = bits - 1; i >= 0; i--) {
        cout << ((u >> i) & 1);              // біт номер i
        if (i % 4 == 0 && i != 0) cout << ' ';
    }
    unsigned long long mask = (1ULL << bits) - 1;   // лише потрібні біти
    cout << "\t0x" << hex << (u & mask) << dec << endl;
}

void inc(int& v) { v++; }

void depth(int n) {
    int local = n;   // у кожного виклику своя local у своєму кадрі стека
    cout << "  depth " << n << ": &local = " << &local << endl;
    if (n > 0) depth(n - 1);
}

int main() {
```

```
    SetConsoleOutputCP(CP_UTF8);   // консоль у UTF-8 (разом із ключем /utf-8 у проєкті)

    cout << "== 1. Адреси ==" << endl;
    int x = 42;
    cout << "x = " << x << ", &x = " << &x << endl;
    cout << "sizeof(int) = " << sizeof(int) << ", sizeof(int*) = " << sizeof(int*) << endl;

    unsigned int num = 0x12345678;
    unsigned char* bytes = (unsigned char*)&num;
    cout << "байти 0x12345678 у пам'яті: " << hex;
    for (int i = 0; i < 4; i++) cout << (int)bytes[i] << " ";
    cout << dec << endl;

    cout << "\n== 2. Вказівники ==" << endl;
    int* p = &x;
    *p = 20;
    cout << "після *p = 20: x = " << x << endl;
    inc(x);
    cout << "після inc(x): x = " << x << endl;

    int a[4] = { 10, 20, 30, 40 };
    int* q = a;
    cout << "q = " << q << ", q + 1 = " << q + 1 << endl;
    cout << "*(q + 2) = " << *(q + 2) << ", a[2] = " << a[2] << endl;

    cout << "\n== 3. Стек ==" << endl;
    depth(3);   // адреси local зменшуються: стек росте вниз

    cout << "\n== 4. Купа ==" << endl;
    int* h = new int(7);
    cout << "h = " << h << ", *h = " << *h << endl;
    delete h;

    int* arr = new int[5];
    arr[0] = 1;
    cout << "arr = " << arr << " (купа), &arr = " << &arr << " (стек)" << endl;
    delete[] arr;

    cout << "\n== 5. Бітові операції ==" << endl;
    unsigned char m = 12, n = 10;   // 0000 1100 і 0000 1010
```

```
    printBits("a", m, 8);
    printBits("b", n, 8);
    printBits("a & b", m & n, 8);
    printBits("a | b", m | n, 8);
    printBits("a ^ b", m ^ n, 8);
    printBits("~a", (unsigned char)~m, 8);
    printBits("a << 1", m << 1, 8);
    printBits("a >> 2", m >> 2, 8);
    printBits("-88", -88, 8);       // додатковий код
    printBits("-1", -1, 32);        // самі одиниці

    cout << endl;
    unsigned int f = 74;            // 0100 1010
    printBits("x", f, 8);
    printBits("set 2", f | (1 << 2), 8);      // встановити біт 2
    printBits("clear 3", f & ~(1 << 3), 8);   // скинути біт 3
    printBits("toggle 6", f ^ (1 << 6), 8);   // інвертувати біт 6
    cout << "біт 1 = " << ((f >> 1) & 1) << endl;

    cout << endl;
    unsigned int nums[4] = { 1, 6, 16, 24 };
    for (int i = 0; i < 4; i++) {
        unsigned int k = nums[i];
        printBits("k", k, 8);
        if ((k & (k - 1)) == 0) cout << "  -> степінь двійки" << endl;
    }

    cout << "\n== Прапорці ==" << endl;
    const unsigned int READ = 1, WRITE = 2, EXEC = 4;   // біти 0, 1, 2
    unsigned int perm = READ | WRITE;
    printBits("perm", perm, 4);
    cout << "можна писати: " << ((perm & WRITE) != 0) << endl;
    perm = perm & ~WRITE;
    printBits("без WRITE", perm, 4);
    cout << "можна виконувати: " << ((perm & EXEC) != 0) << endl;

    cout << "\n== Колір RGB ==" << endl;
    unsigned int c = 0xFF8040;
    unsigned int r = (c >> 16) & 0xFF;
    unsigned int g = (c >> 8) & 0xFF;
    unsigned int b = c & 0xFF;
```

```
    printBits("c", c, 24);
    cout << "R = " << r << ", G = " << g << ", B = " << b << endl;

    cout << "\n== Маска підмережі ==" << endl;
    unsigned int ip = (192u << 24) | (168u << 16) | (1u << 8) | 37u;   // 192.168.1.37
    unsigned int mask = 0xFFFFFF00;                                    // 255.255.255.0
    unsigned int net = ip & mask;
    printBits("ip", ip, 32);
    printBits("mask", mask, 32);
    printBits("net", net, 32);
    cout << "мережа: " << (net >> 24) << "." << ((net >> 16) & 0xFF) << "."
         << ((net >> 8) & 0xFF) << "." << (net & 0xFF) << endl;

    return 0;
}
```